LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 5
Student Name: Nimidithalli Nandu
Roll No.: A24126552100
Date: 20-09-2026

---

1. TITLE

Develop a Student Server Using Express.js and Implement Multiple Routes

---

2. AIM

To develop a basic web server using Node.js and Express.js that implements multiple routes for displaying a welcome message, student information in JSON format, and application information

---

3. OBJECTIVE

The objectives of this experiment are:
- To understand the basics of Node.js and Express.js.
- To create a basic Express.js server.
- To understand HTTP GET requests.
- To implement multiple routes using Express.js.
- To return text responses from an Express server.
- To return structured student data in JSON format.
- To understand route handling using app.get().
- To start and run an Express server on a specified port.

---

4. THEORY

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser. It is commonly used for developing server-side applications.

Express.js is a lightweight web framework built on Node.js. It simplifies the development of web servers and provides methods for handling HTTP requests and routes.

In this application, Express.js is used to create a simple Student Server with three different routes.

The application implements the following routes:

● **GET /** – Displays a welcome message.
● **GET /students** – Returns a list of student details in JSON format.
● **GET /about** – Displays information about the Student Server.

The app.get() method is used to handle GET requests for each route.

The res.send() method sends a text response to the client, while res.json() sends JavaScript objects or arrays as JSON data.

The basic flow of the application is:

Client Request → Express Server → Route Matching → Response → Client

The server runs on **port 3000** and can be accessed using:

http://localhost:3000

---

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
2. Install the Express.js package using npm.
3. Import the Express module using require("express").
4. Create an Express application using express().
5. Define the server port as 3000.
6. Create the home route / using app.get().
7. Send a welcome message as the response for the home route.
8. Create the /students route using app.get().
9. Create an array containing student objects.
10. Store student ID, name and course information in each object.
11. Use res.json() to send the student array as JSON.
12. Create the /about route using app.get().
13. Send information about the Student Server using res.send().
14. Start the Express server using app.listen().
15. Display the server URL in the console.

16. Open the routes in a web browser.
17. Test each route separately.
18. Verify the actual output against the expected output.

---

6. PROGRAM

Code.js

```
const express = require("express");

const app = express();

const PORT = 3000;


// Home route

app.get("/", (req, res) => {

    res.send("Welcome to the Student Server");

});


// Students route

app.get("/students", (req, res) => {

    const students = [

        {
            id: 1,
            name: "Rahul",
            course: "CSE"
        },

        {
            id: 2,
```

```
            name: "Priya",
            course: "AI & ML"
        },

        {
            id: 3,
            name: "Arjun",
            course: "ECE"
        },

        {
            id: 4,
            name: "Sneha",
            course: "CSE"
        },

        {
            id: 5,
            name: "Kiran",
            course: "IT"
        }

    ];

    res.json(students);

});


// About route

app.get("/about", (req, res) => {

    res.send(
        "Student Server is a basic Express.js application " +
        "created to demonstrate routing and HTTP methods."
    );

});
```

```
// Start server

app.listen(PORT, () => {

    console.log(`Server running at http://localhost:${PORT}`);

});
```

---

7. CODE EXPLANATION

Code / Function	Explanation
`require("express")`	Imports the Express.js module.
`express()`	Creates an Express application.
`PORT = 3000`	Defines port 3000 for running the server.
`app.get()`	Handles HTTP GET requests for a particular route.
`"/"`	Represents the home route.
`"/students"`	Represents the route used to access student information.
`"/about"`	Represents the route containing application information.
`res.send()`	Sends a text response to the client.
`res.json()`	Sends data in JSON format to the client.
`students`	Stores an array of student objects.

`id`	Represents the unique ID of a student.
`name`	Stores the student's name.
`course`	Stores the student's course.
`app.listen()`	Starts the Express server on the specified port.
`console.log()`	Displays the server URL in the terminal.

---

8. TEST CASES AND EXECUTION DATA

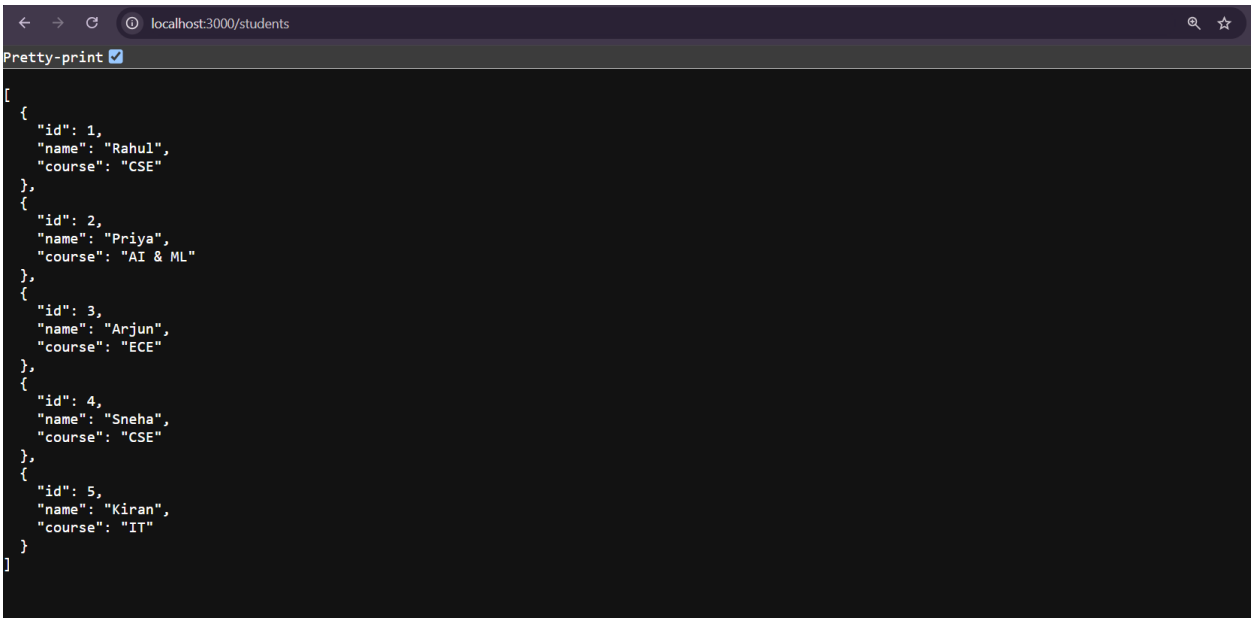
Test Case	URL / Route	Expected Result	Actual Result	Status
TC-01	`/`	Welcome message should be displayed	Welcome message displayed	Pass
TC-02	`/students`	Student details should be returned in JSON format	Student details displayed in JSON format	Pass
TC-03	`/about`	About message should be displayed	About message displayed	Pass

TC-01 output:

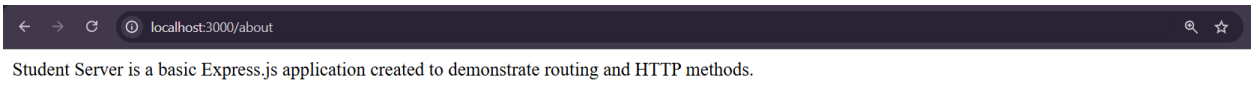
```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Expressjs_Student_Server> node co
```

Welcome to the Student Server

TC-02 Output:



TC_03 Output:



---

9. OUTPUT:

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Expressjs_Student_Server> node code.js
Server running at http://localhost:3000
```

---

10. RESULT

Thus, a **Student Server** was successfully developed using **Node.js and Express.js**. Multiple GET routes were implemented to handle the home page, student information and about page. Student data was successfully returned in JSON format, and all routes were tested successfully.
